

Tema: Programación de microcontroladores: anti-rebote desarrollado por MEF para garantizar detección positiva de pulsadores mecánicos, aplicación en STM32CubeIDE, programación de microcontroladores.

Tipo de Actividad de Formación Práctica: Laboratorio

Unidad Temática 2.- Arquitectura de un microprocesador.

Unidad Temática 6.- Microprocesadores.

Trabajo de Laboratorio: Implementación de un driver de funciones anti-rebote (debounce) mediante MEF, aplicado en proyectos con STM32CubeIDE y placas de desarrollo STM32NUCLEO-F4XX.

1. Alcance

El objetivo de este trabajo de laboratorio es que los estudiantes logren:

- Aplicar el concepto de Máquina de Estados Finitos (MEF) y el uso de drivers para implementar funciones anti-rebotes por software en STM32CubeIDE.
- Implementar un driver de funciones debounce y aplicarlo en proyectos previos. • Integrar el driver en aplicaciones existentes, modificando el código para utilizar retardos no bloqueantes y funciones estructuradas.
- Presentar el trabajo en laboratorio con funcionamiento correcto y defensa técnica de lo realizado.

2. Elementos / Instrumental a Utilizar

- Placa de desarrollo STM32-NUCLEO-F4XX (modelos sugeridos: F401-RE, F412-ZG, F413-ZH, F429-ZI).
- Pulsadores mecánicos conectados a entradas digitales.
- LEDs para pruebas de encendido y apagado controlados por pulsadores.
- Computadora con software de desarrollo integrado STM32CubeIDE instalada.
- Software de Programación y *Debugging*: STM32CubeIDE para programar y depurar el código.
- Repositorio GitHub grupal para subir las aplicaciones desarrolladas.
- Documentación técnica del fabricante y material de la cátedra sobre MEF y debounce.

3. Actividades a- Implementación del driver:

- Partir de un proyecto base de la práctica anterior sobre programación de microcontroladores e implementar en main.c y main.h las funciones necesarias para el sistema anti-rebote con un retardo no bloqueante de 40 ms.
- Definir el tipo de datos *debounceState_t* y las funciones **debounceFSM_init()**, **debounceFSM_update()**, **buttonPressed()** y **buttonReleased()**.
- Configurar **buttonPressed()** para invertir el estado del LED1 y **buttonReleased()** para invertir el estado del LED3.

b- Implementación modular:

- Crear el módulo API debounce en /Drivers/API/src y /Drivers/API/inc con sus funciones públicas y privadas.
- Agregar la función *bool_t readKey()* que detecta flancos descendentes y reinicia el estado al ser leída.

c- Integración en aplicaciones previas:

- Modificar las Apps de la práctica anterior para usar el driver de debounce y retardos no bloqueantes del módulo API delay.

d- Pruebas y verificación:

- Compilar, depurar y probar en laboratorio cada aplicación modificada.
- Evaluar la mejora en el comportamiento de las aplicaciones con el uso del driver debounce.

e- Informe de laboratorio:

El informe debe presentar los siguientes puntos:

- Introducción al uso de debounce por MEF y ventajas en aplicaciones comerciales.
- Lista de aplicaciones modificadas, autores y observaciones.
- *Link al repositorio grupal con el código final.*

4. Duración: 5 Hs.

Introducción: Implementación de funciones Antirebote utilizando máquinas de estado finito (MEF) con el entorno de desarrollo STM32CubeIDE.

En sistemas embebidos, el manejo eficiente de las entradas digitales, como botones o interruptores, es crucial para garantizar la estabilidad y confiabilidad de una aplicación. Un problema común al leer el estado de un botón es el fenómeno conocido como "rebote" (debounce), el cual ocurre cuando una pulsación genera múltiples transiciones de estado debido a la naturaleza mecánica del botón. Este comportamiento puede causar lecturas erróneas y resultados no deseados en el software.

Para resolver este problema, las funciones antirebote implementadas mediante Máquinas de Estado Finito (MEF) ofrecen una solución robusta y eficiente. Una MEF divide el proceso de gestión del botón en estados definidos (por ejemplo, `BUTTON_UP`, `BUTTON_FALLING`, `BUTTON_DOWN`, `BUTTON_RISING`) y establece transiciones claras entre ellos basadas en condiciones específicas y temporizadores. Este enfoque permite filtrar de manera determinista los rebotes al validar que una transición entre estados sea consistente durante un tiempo definido.



Ventajas del uso de MEF para funciones antirebote:

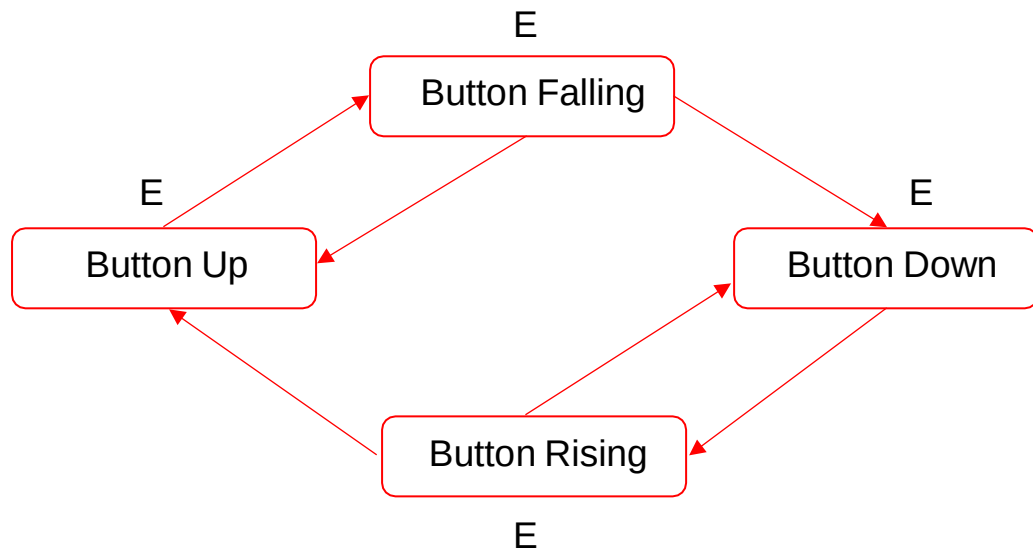
- **Flexibilidad:** Las MEF permiten modelar una amplia variedad de comportamientos, adaptándose a diferentes escenarios y requisitos.
- **Fiabilidad:** Al definir estados claros y transiciones precisas, se reduce la posibilidad de errores en la detección de pulsaciones.
- **Facilidad de mantenimiento:** La estructura modular de las MEF facilita la comprensión y modificación del código.
- **Determinismo:** El comportamiento de la MEF es predecible y reproducible, lo que facilita la depuración y el análisis del sistema.
- **Escalabilidad:** Las MEF pueden ser fácilmente extendidas para gestionar múltiples botones o eventos.

Funcionamiento general del algoritmo:

El algoritmo propuesto se basa en una MEF que define cuatro estados posibles para el botón:

1. **BUTTON_UP:** El botón está en reposo.
2. **BUTTON_FALLING:** Se ha detectado un flanco descendente, se inicia un temporizador para filtrar rebotes.
3. **BUTTON_DOWN:** El botón se considera presionado de forma válida.

4. **BUTTON_RISING**: Se ha detectado un flanco ascendente, se inicia un temporizador para filtrar rebotes.



API_Debounce:

Las funciones para implementar la MEF y lograr evitar los problemas debido al rebote estarán en la API_Debounce.

Dentro del archivo API_Debounce.h encontraremos las declaraciones de las funciones y las variables utilizadas, además de la estructura de enumeración para la MEF (debounceState_t). Las funciones utilizadas y que se desarrollan dentro de API_Debounce.c son:

- debounceFSM_init: Se encarga de inicializar la función con los valores predeterminados. Configura el estado inicial de la MEF como BUTTON_UP, por un lado, indicando donde comenzara la MEF en la función debounceFSM_update y que el botón es activo bajo. Inicializa un temporizador DEBOUNCE_DELAY que se utiliza para filtrar los rebotes mecánicos, tanto los producidos en los flancos descendentes como los ascendentes. Por último, apaga todos los leds de la placa de desarrollo por precaución.
- debounceFSM_update: Esta función recibe como parámetro el estado del botón que se desea utilizar y se actualiza de forma periódica dentro del programa principal. Dentro de esta función se implementa la máquina de estado y según el estado en que se encuentre podremos utilizar una variable auxiliar, en este caso implementamos la variable booleana keyPressed, la cual indicara si se presionó o no el botón. Se utiliza un delay de 40ms en cada estado para evitar falsas lecturas debido al rebote mecánico del botón.
- ReadKey: Función encargada de verificar si el pulsador fue presionado. Devuelve una variable booleana keyPress. Si el valor de retorno es true, se entiende que hubo un evento de pulsación válido. Por otro lado, reinicia el estado interno de la variable keyPressed, para evitar lecturas erróneas/repetitivas de la misma pulsación. Además, también se reinicia el estado de la variable que detecta un flanco descendente por la misma razón.

Aplicaciones desarrolladas:

Se implemento el driver API_debounce, el cual cumple la función de anti-rebote de un pulsador, los cambios en las aplicaciones se realizaron de modo que se inicialicen las funciones correspondientes de la API (debounceFSM_init) y con la función debounceFSM_update mediante el uso de máquina de estado se evitan las pulsaciones no deseadas. La función readKey será la utilizada para evaluar la condición de si se presionó o no el botón.

App 4.1: En esta aplicación implementando API_debounce, se alternará entre 2 estados cuando se presione el botón integrado de la placa. Como estado inicial se tiene el encendido y apagado en secuencia de los leds cada 200ms (verde, azul y rojo) y al presionar el botón alterna al otro estado donde solo parpadea cada 200ms el led azul (LED2).

Para detectar que el botón fue presionado y ejecutar el cambio de estado o secuencia, se utilizó la función readKey como condición de un "if" para realizar el cambio de secuencia y restablecer las condiciones iniciales necesarias en cada una.

Permalink: [main.c](#)

App 4.2: En esta aplicación se debe alternar entre dos secuencias cuando se presiona el botón, por lo que se implementó de igual manera que en la aplicación 4.1 una condición if(readKey) para alternar entre las secuencias dentro de un switch.

La primera secuencia enciende y apaga los leds cada 200ms en orden (verde, azul y rojo), la segunda secuencia hará lo mismo, pero en orden inverso (rojo, azul y verde).

Permalink: [main.c](#)

App 4.3: De igual manera que en las aplicaciones anteriores se implementó una condición con la función readKey para evaluar si se presionó el botón o no y cambiar entre 4 secuencias distintas para el encendido de los leds. En esta aplicación fue donde más se pudo notar el efecto del driver anti-rebote, ya que en versiones anteriores era común que se saltee una o más secuencias al presionar el botón ya que se detectaba como si esta se presionó muchas veces.

Permalink: [main.c](#)

App 4.4: Al igual que en la aplicación 4.3 se alterna entre cuatro secuencias donde cada una hace parpadear los leds en simultánea, pero con diferentes intervalos de tiempo. La implementación de la API_debounce se realizó de la misma forma que en las otras aplicaciones.

Permalink: [main.c](#)

En general para todas las aplicaciones no hubo grandes dificultades para realizar la implementación del driver, sino más bien en la creación de este, entendiendo como es el funcionamiento de la máquina de estado y la lógica para resolver los diferentes estados de la misma evitando los rebotes mecánicos al detectar el estado de un pulsador.

Repositorio Grupo 6: [AFP_5_Grupo_6_2025](#)

Enlaces

Repositorio del Grupo 6:

[Grupo 6 TDII 2025/AFP 5 GRUPO 6 TDII 2025 at master · ZePhyR1906/Grupo 6 TDII 2025](#)

App 1.1:

[Grupo 6 TDII 2025/AFP 5 GRUPO 6 TDII 2025/App 1 1 GRUPO 6 2025 at master · ZePhyR1906/Grupo 6 TDII 2025](#)

App 1.2:

[Grupo 6 TDII 2025/AFP 5 GRUPO 6 TDII 2025/App 1 2 GRUPO 6 2025 at master · ZePhyR1906/Grupo 6 TDII 2025](#)

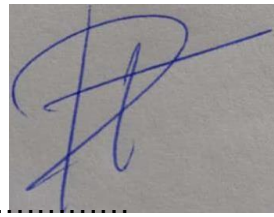
App 1.3:

[Grupo 6 TDII 2025/AFP 5 GRUPO 6 TDII 2025/App 1 3 GRUPO 6 2025 at master · ZePhyR1906/Grupo 6 TDII 2025](#)

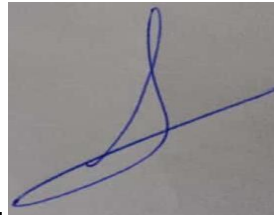
App 1.4:

[Grupo 6 TDII 2025/AFP 5 GRUPO 6 TDII 2025/App 1 4 GRUPO 6 2025 at master · ZePhyR1906/Grupo 6 TDII 2025](#)

ESTUDIANTE: Tejerina Damián Federico.....



ESTUDIANTE:.....Leal Franco Miquel.....



ESTUDIANTE:Porcel Guillermo Osvaldo.....



FECHA DE INICIO: 17/09/2025

FECHA DE PRESENTACIÓN: 10/10/2025

CONFORMIDAD DEL DOCENTE:.....